

Titanic 생존자 예측 머신러닝 파이프라인

결측치 처리 · 인코딩 · 스케일링 · 파생변수 · Feature Selection 비교 실험 보고서

데이터셋: Titanic - Machine Learning from Disaster (Kaggle)

작업 유형: 이진 분류 (생존 여부 예측)

사용 모델: Logistic Regression · Random Forest · Gradient Boosting

소스코드: GitHub 저장소 링크로 대체 (Titanic_ML_Pipeline.ipynb)

1. 데이터셋 소개

목적 — 승객의 인적정보(성별·나이·객실등급 등)와 탑승 정보를 바탕으로 타이타닉 침몰 사고에서의 **생존 여부(Survived)**를 예측하는 지도학습 기반 이진 분류(Classification) 문제입니다. 출처: [Kaggle Titanic Competition](#)

데이터 shape

파일	행(샘플) 수	열(컬럼) 수
train.csv	891	12
test.csv	418	11 (Survived 제외)

필수 조건 충족 — 지도학습(라벨 존재) ✓ | 수치형+범주형 혼합 ✓ | 샘플 수 891개 (500개 이상) ✓

컬럼 설명 표

컬럼	설명	타입
PassengerId	승객 고유 ID	수치형(ID)
Survived	생존 여부 (0=사망, 1=생존) — 타겟 변수	범주형(이진)
Pclass	객실 등급 (1=1등석, 2=2등석, 3=3등석)	범주형(순서)
Name	승객 이름 (Title 파생변수 추출에 사용)	텍스트
Sex	성별	범주형
Age	나이 (177건 결측)	수치형
SibSp	동승한 형제자매/배우자 수	수치형
Parch	동승한 부모/자녀 수	수치형
Ticket	티켓 번호	텍스트
Fare	운임	수치형
Cabin	객실 번호 (687건 결측, 77.1%)	텍스트
Embarked	승선 항구 (C=Cherbourg, Q=Queenstown, S=Southampton)	범주형

2. 탐색적 데이터 분석 (EDA)

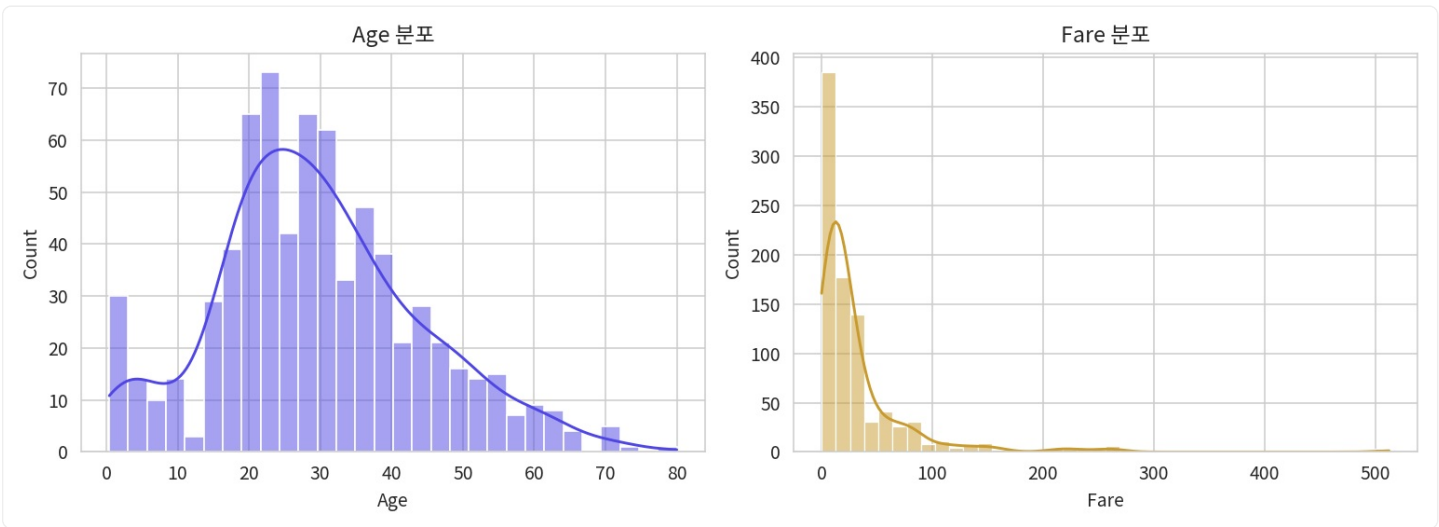
결측치 비율 분석

변수	결측개수	결측비율(%)
Cabin	687	77.100
Age	177	19.870
Embarked	2	0.220

이상치 탐색 (IQR 방식, $1.5 \times IQR$ 기준)

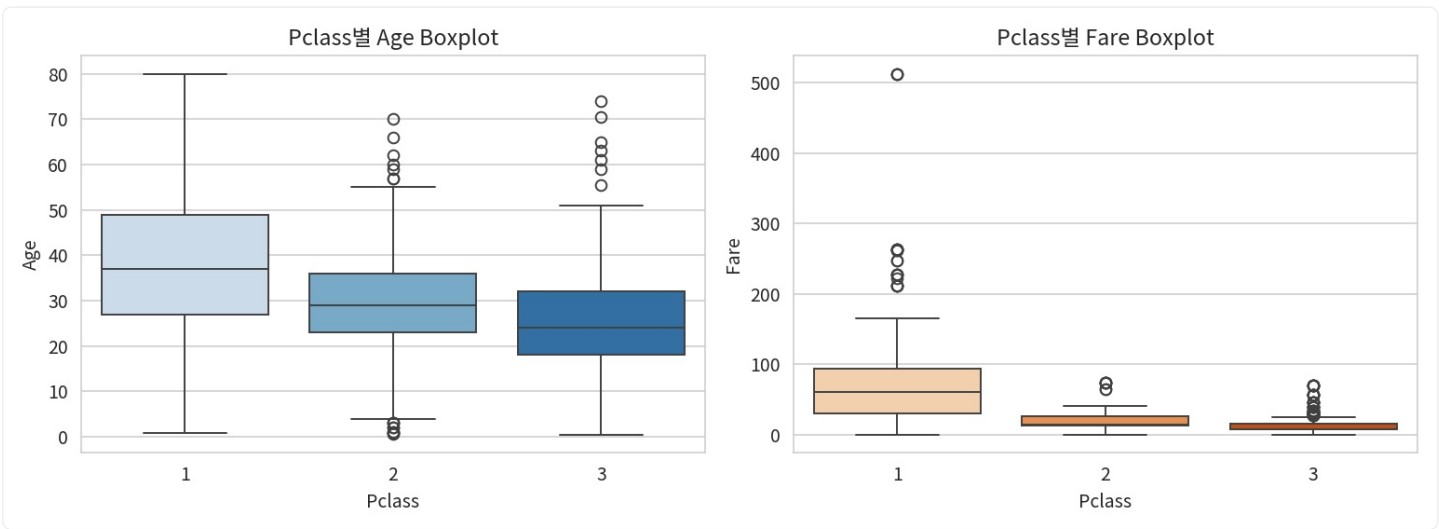
변수	이상치개수	이상치비율(%)	하한	상한
Age	11	1.230	-6.690	64.810
Fare	116	13.020	-26.720	65.630
SibSp	46	5.160	-1.500	2.500
Parch	213	23.910	0.000	0.000

변수 분포 시각화 (Histogram)



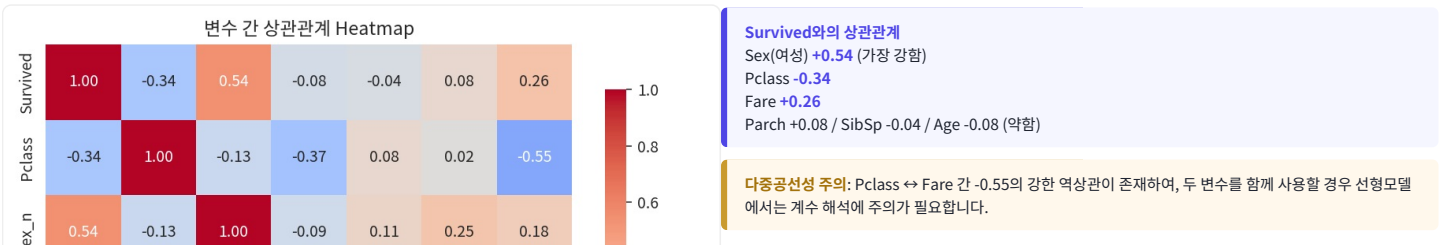
Age는 20~30대에 집중된 우측 비대칭 분포, Fare는 저가 운임에 치우친 극단적 우측 비대칭(Long-tail) 분포를 보입니다.

이상치 시각 확인 (Boxplot)



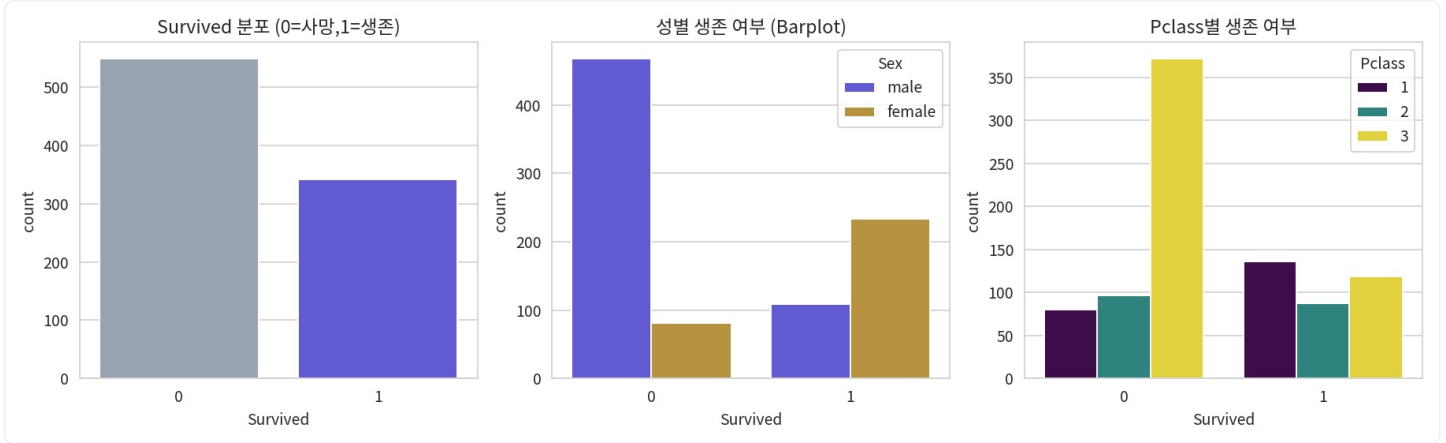
1등석일수록 평균 연령이 높고 운임 분산이 크며, 상위 이상치(고가 티켓)가 집중되어 있습니다.

상관관계 분석 (Heatmap)





타겟 변수 분포 (Countplot / Barplot)



생존 38.4% vs 사망 61.6% (약 1.6:1, 경미한 불균형) · 여성 생존율 74.2% vs 남성 18.9% · 1등석 63.0% vs 3등석 24.2%

EDA 분석 결과 요약

데이터 품질 문제

Cabin은 결측률 77.1%로 원본 그대로 사용이 어려워 '객실정보 보유 여부(HasCabin)' 파생변수로 대체했습니다. Age는 19.9% 결측으로 대체 전략 비교가 반드시 필요하며, Embarked는 2건뿐이라 최빈값 대체로 충분합니다. Fare의 상위 13%는 이상치로 분류되지만 1등석 고가 티켓이라는 실제 현상이므로 제거 대신 Robust 계열 스케일링으로 완화하는 것이 타당합니다.

불균형 여부

생존(1) 38.4% / 사망(0) 61.6%로 약 1.6:1 수준의 경미한 불균형이며, 심각한 클래스 불균형(예: 1:10 이상) 수준은 아니어서 별도의 오버/언더샘플링 없이 표준 분류 모델로 충분히 대응 가능하다고 판단했습니다.

주요 변수 특징

Sex가 생존 여부를 가장 강하게 설명하는 변수이며, Pclass·Fare가 그 뒤를 잇습니다. 반면 Age, SibSp, Parch는 단독 상관관계가 약해($|r|<0.1$), 파생변수(FamilySize, FarePerPerson 등)를 통한 정보 보강이 필요함을 시사합니다.

3. Feature Engineering 과정

3-4. 파생 변수 생성 (6개)

파생변수	생성 방법	의도
Title	Name에서 정규식으로 호칭 추출 (Mr/Mrs/Miss/Master/Rare)	나이·성별·사회적 지위를 압축한 강력한 신호
FamilySize	SibSp + Parch + 1	가족 규모에 따른 생존 전략 차이 반영
IsAlone	FamilySize == 1 여부	혼자 탑승한 승객의 생존 패턴 구분
FarePerPerson	Fare / FamilySize	가족 단위 운임을 1인 기준으로 정규화 (실질 경제력 반영)
HasCabin	Cabin 결측 여부(0/1)	77% 결측인 Cabin 원본 대신 정보 유무만 활용
AgeGroup	Age를 [0,12,18,35,60,120] 구간으로 범주화	연령대별 비선형 생존 패턴 포착

필수 조건 충족 — 파생변수 2개 이상 생성 조건을 6개로 초과 달성 ✔

3-1. 결측치 처리 비교 (Ablation A)

고정 조건: 인코딩=One-Hot, 스케일링=Standard, Feature Selection=미적용, 파생변수=적용. 결측치 전략만 변경하여 순수 효과를 분리했습니다.

결측치전략	모델	Accuracy	Precision	Recall	F1	ROC_AUC
mean	LogisticRegression	0.832	0.800	0.754	0.776	0.876
mean	RandomForest	0.799	0.732	0.754	0.743	0.830
mean	GradientBoosting	0.821	0.794	0.725	0.758	0.840
median	LogisticRegression	0.832	0.800	0.754	0.776	0.875
median	RandomForest	0.788	0.725	0.725	0.725	0.829
median	GradientBoosting	0.804	0.774	0.696	0.733	0.836
most_frequent	LogisticRegression	0.832	0.800	0.754	0.776	0.874
most_frequent	RandomForest	0.804	0.743	0.754	0.748	0.828
most_frequent	GradientBoosting	0.804	0.783	0.681	0.729	0.845
drop	LogisticRegression	0.856	0.849	0.789	0.818	0.882
drop	RandomForest	0.784	0.737	0.737	0.737	0.840
drop	GradientBoosting	0.806	0.800	0.702	0.748	0.844

Mean/Median/Most Frequent 간 성능 차이는 0.01~0.03p 이내로 크지 않았습니다. 'drop'(결측 행 삭제)이 Logistic Regression에서는 가장 높은 Accuracy(0.8561)를 보였으나, 이는 샘플 수 감소(177건 중 일부 소실)로 인한 분산 효과일 수 있어 일반화에는 주의가 필요합니다. 실무적으로는 샘플을 보존하면서 성능이 안정적인 **Median** 또는 **Most Frequent Imputation**을 권장합니다.

3-2. 범주형 인코딩 비교 (Ablation B)

고정 조건: 결측치=Median, 스케일링=Standard, Feature Selection=적용. 인코딩 방식만 변경했습니다.

인코딩	모델	Accuracy	Precision	Recall	F1	ROC_AUC
label	LogisticRegression	0.810	0.787	0.696	0.739	0.847
label	RandomForest	0.793	0.750	0.696	0.722	0.845
label	GradientBoosting	0.776	0.746	0.638	0.688	0.833
onehot	LogisticRegression	0.804	0.774	0.696	0.733	0.858
onehot	RandomForest	0.810	0.761	0.739	0.750	0.846
onehot	GradientBoosting	0.782	0.759	0.638	0.693	0.831

Random Forest는 One-Hot이 근소 우위(Accuracy 0.8101 vs 0.7933)였지만, Logistic Regression과 Gradient Boosting은 오히려 Label(Ordinal) Encoding이 비슷하거나 소폭 더 나은 결과를 보였습니다. **One-Hot이 항상 최선은 아니며**, 트리 기반 모델은 Label Encoding만으로도 분기 기준을 충분히 학습할 수 있음을 확인했습니다.

3-3. 스케일링 비교 (Ablation C)

고정 조건: 결측치=Median, 인코딩=One-Hot, Feature Selection=적용. 스케일링 방식만 변경했습니다.

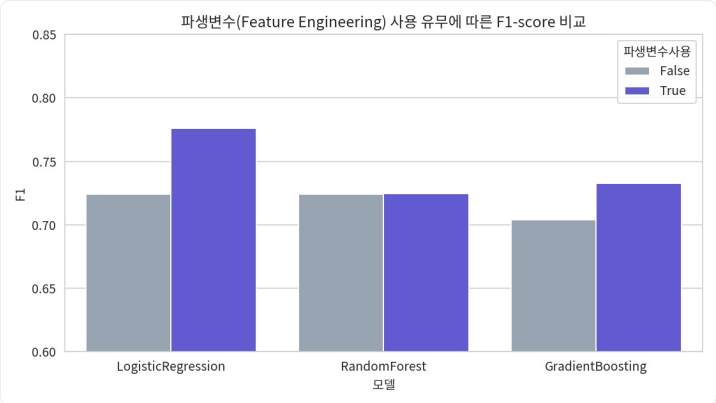


Random Forest·Gradient Boosting은 스케일링 종류와 무관하게 Accuracy가 거의 동일(트리 분할 기준은 척도에 불변)했지만, Logistic Regression은 방식에 따라 0.793~0.805로 변동했습니다. **거리·경사 기반 모델일수록 스케일링에 민감**하다는 이론과 일치하는 결과입니다.

파생변수(Feature Engineering) 기여도 분석 (Ablation E)

고정 조건: 결측치=Median, 인코딩=One-Hot, 스케일링=Standard, Feature Selection=미적용. 파생변수 사용 여부만 변경했습니다.

파생변수사용	모델	Accuracy	Precision	Recall	F1	ROC_AUC
False	LogisticRegression	0.804	0.793	0.667	0.724	0.843
False	RandomForest	0.804	0.793	0.667	0.724	0.835
False	GradientBoosting	0.793	0.786	0.638	0.704	0.819
True	LogisticRegression	0.832	0.800	0.754	0.776	0.875
True	RandomForest	0.788	0.725	0.725	0.725	0.829
True	GradientBoosting	0.804	0.774	0.696	0.733	0.836

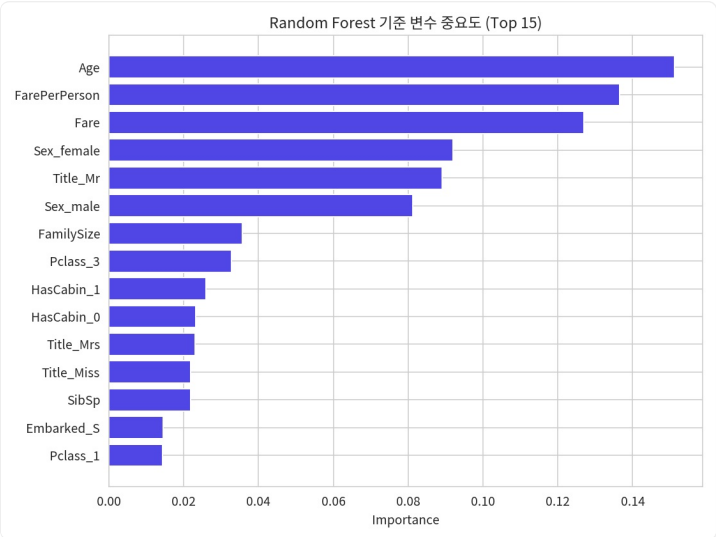


Logistic Regression의 F1-score가 0.7244→0.7761로 가장 크게 향상되었고, Gradient Boosting도 0.7040→0.7328로 개선되었습니다. 반면 Random Forest는 F1이 거의 동일(0.7244→0.7246)하고 Accuracy는 0.8045→0.7877로 소폭 하락했는데, 이는 트리 모델이 원본 변수 조합만으로도 유사한 상호작용 패턴을 일부 학습할 수 있기 때문으로 해석됩니다. 종합적으로 **선형 모델일수록 파생변수의 효과가 크게 나타났습니다**.

4. 변수 선택 (Feature Selection)

Random Forest Feature Importance 기반 상위 10개 변수 선정

변수	중요도
Age	0.151
FarePerPerson	0.136
Fare	0.127
Sex_female	0.092
Title_Mr	0.089
Sex_male	0.081
FamilySize	0.036
Pclass_3	0.033
HasCabin_1	0.026
HasCabin_0	0.023

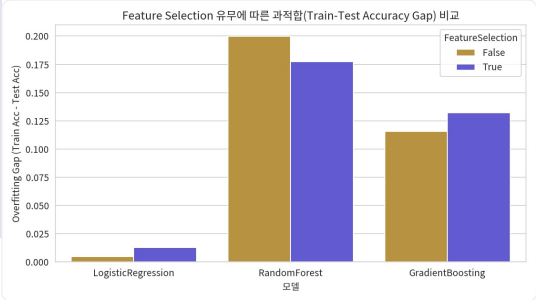


Age, FarePerPerson, Fare, Sex, Title_Mr가 상위권을 차지했으며, 특히 **FarePerPerson(파생변수)**이 2위를 기록해 Feature Engineering의 실질적 기여를 뒷받침합니다.

Feature Selection 적용 전/후 성능 비교 + 과적합(Overfitting) 분석

고정 조건: 결측치=Median, 인코딩=One-Hot, 스케일링=Standard. Train_Accuracy - Test_Accuracy = Overfit_Gap으로 과적합 정도를 정량화했습니다.

FeatureSelection	모델	Accuracy	Precision	Recall	F1	ROC_AUC	Train_Accuracy	Overfit_Gap
False	LogisticRegression	0.832	0.800	0.754	0.776	0.875	0.837	0.005
False	RandomForest	0.788	0.725	0.725	0.725	0.829	0.987	0.200
False	GradientBoosting	0.804	0.774	0.696	0.733	0.836	0.920	0.116
True	LogisticRegression	0.804	0.774	0.696	0.733	0.858	0.817	0.013
True	RandomForest	0.810	0.761	0.739	0.750	0.846	0.987	0.177
True	GradientBoosting	0.782	0.759	0.638	0.693	0.831	0.914	0.132



Random Forest는 Feature Selection 적용 후 Overfit Gap이 0.1997→0.1773로 감소하고 Test Accuracy도 0.7877→0.8101로 개선되어 **과적합 완화 + 성능 향상**을 동시에 달성했습니다. 반면 Gradient Boosting은 Gap이 0.1155→0.1322로 소폭 증가해, Feature Selection의 효과가 **모델 구조에 따라 다르게 나타남**을 확인했습니다 (RF 기반 중요도로 선정한 피쳐 집합이 모든 모델에 동일하게 최적은 아님).

5. 모델 학습 및 평가 과정

학습/평가 방법론

- train.csv(891건)를 **Stratified 80/20 분할**하여 train(712) / holdout test(179)로 구성 (random_state=42)
- test.csv는 실제 Survived 라벨이 없는 Kaggle 비공개 평가셋이므로, 모델 성능 비교는 holdout test 기준으로 수행
- 결측치 대체 통계량(평균/중앙값/최빈값)은 **train 기준으로만 계산 후 test에 동일 적용**하여 Data Leakage 방지
- 전처리~모델까지 전 과정을 sklearn Pipeline/ColumnTransformer 로 구성 (가산점 항목)

사용 모델 (3개, 최소 2개 조건 초과 충족)

모델	특징
Logistic Regression	해석 가능한 선형 분류기, 스케일링에 민감
Random Forest	배깅 기반 트리 앙상블, 비선형 상호작용 포착에 강함
Gradient Boosting	부스팅 기반 트리 앙상블 (xgboost/lightgbm 대체: 실행환경에 해당 패키지 미설치로 sklearn 내장 모델 사용)

GridSearchCV 하이퍼파라미터 튜닝 (가산점)

대상: Exp-2 설정(Median+One-Hot+Standard+Feature Selection)의 Random Forest, 5-Fold StratifiedKFold, scoring=F1

항목	값
탐색 파라미터	n_estimators=[200,400], max_depth=[4,6,8,None], min_samples_leaf=[1,2,4]
Best Params	{'clf__max_depth': None, 'clf__min_samples_leaf': 4, 'clf__n_estimators': 200}
Best CV F1 (5-Fold 평균)	0.7823
Holdout Test 성능	Accuracy 0.7877 / F1 0.7077 / ROC-AUC 0.8431

관찰: CV F1(0.7823)과 holdout F1(0.7077) 간 차이가 존재합니다. holdout 샘플이 179건으로 적어 표본 변동성의 영향을 받을 수 있으며, 이는 단일 split 평가의 한계를 보여주는 사례입니다.

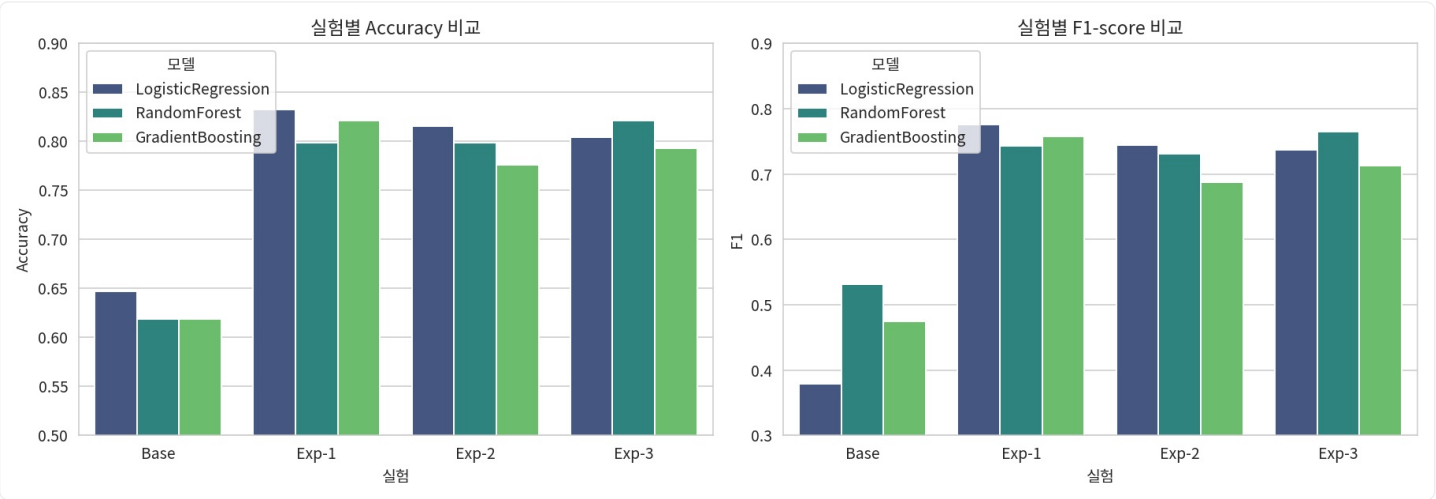
6. 실험 비교 항목 (성능 비교표)

실험 설정

실험	결측치처리	인코딩	스케일링	파생변수	FeatureSelection
Base	drop	none	none	False	False
Exp-1	mean	onehot	standard	True	False
Exp-2	median	label	minmax	True	True
Exp-3	most_frequent	onehot	robust	True	True

실험 × 모델별 성능 비교

실험	모델	Accuracy	Precision	Recall	F1	ROC_AUC
Base	LogisticRegression	0.647	0.682	0.263	0.380	0.680
Base	RandomForest	0.619	0.536	0.526	0.531	0.647
Base	GradientBoosting	0.619	0.545	0.421	0.475	0.641
Exp-1	LogisticRegression	0.832	0.800	0.754	0.776	0.876
Exp-1	RandomForest	0.799	0.732	0.754	0.743	0.830
Exp-1	GradientBoosting	0.821	0.794	0.725	0.758	0.840
Exp-2	LogisticRegression	0.816	0.800	0.696	0.744	0.852
Exp-2	RandomForest	0.799	0.754	0.710	0.731	0.848
Exp-2	GradientBoosting	0.776	0.746	0.638	0.688	0.834
Exp-3	LogisticRegression	0.804	0.766	0.710	0.737	0.851
Exp-3	RandomForest	0.821	0.776	0.754	0.765	0.844
Exp-3	GradientBoosting	0.793	0.767	0.667	0.713	0.836



Base(전처리 없음)는 Accuracy 0.62~0.65에 그쳤으나, 전처리/파생변수를 적용한 Exp-1~3은 모두 0.78~0.83으로 크게 향상되었습니다. 세 실험(Exp-1~3) 간 우열은 모델에 따라 엇갈려, 단일 최적 조합은 존재하지 않으며 모델 별 특성에 맞는 전처리 선택이 중요함을 보여줍니다. 전체 12개 조합 중 최고 성능은 Exp-1 × Logistic Regression (Accuracy 0.8324, F1 0.7761, ROC-AUC 0.8756)이었습니다.

7. 최종 결론

Q1. 어떤 전처리 전략이 가장 효과적이었는가?

결측치 처리 방식(Mean/Median/Most Frequent) 자체의 차이보다, **결측치를 채우는 행위와 Sex 등 핵심 범주형 변수를 인코딩해 모델에 포함시키는지 여부**가 압도적으로 더 큰 영향을 미쳤습니다 (Base 대비 Accuracy +0.15~0.19p). 즉 "어떤 통계량으로 채우는가"보다 "결측 처리와 인코딩을 누락 없이 수행하는가"가 핵심이었습니다.

Q2. One-Hot Encoding이 항상 좋은가?

아닙니다. Random Forest에서는 One-Hot이 근소 우위였지만, Logistic Regression·Gradient Boosting에서는 Label(Ordinal) Encoding이 비슷하거나 더 나은 경우도 있었습니다. 트리 기반 모델은 Label Encoding만으로도 분기 기준을 학습할 수 있어, 차원이 늘어나는 One-Hot이 항상 유리한 것은 아니라는 결론을 얻었습니다.

Q3. Feature Selection이 과적합 감소에 기여했는가?

Random Forest에서는 Train-Test Accuracy Gap이 0.1997→0.1773으로 줄고 Test 성능도 개선되어 과적합 완화에 기여했습니다. 다만 Gradient Boosting에서는 Gap이 소폭 증가해, **효과가 모델에 따라 다르게 나타난다**는 점을 확인했습니다.

Q4. 스케일링이 모델별로 어떤 영향을 미쳤는가?

트리 기반 모델(Random Forest, Gradient Boosting)은 스케일링 방식과 무관하게 성능이 거의 동일했지만, Logistic Regression은 방식에 따라 Accuracy가 변동했습니다. **거리·경사 기반 선형모델일수록 스케일링에 더 민감**하다는 이론적 기대와 일치했습니다.

Q5. Feature Engineering이 실제 성능 향상에 얼마나 기여했는가?

Logistic Regression의 F1-score가 파생변수 적용으로 0.7244→0.7761까지 향상되었고, FarePerPerson은 Random Forest 기준 변수 중요도 2위를 기록했습니다. **선형 모델일수록 파생변수의 기여가 크게 나타났으며**, 트리 모델은 원본 변수만으로도 일부 상호작용을 스스로 학습할 수 있어 상대적으로 기여도가 작게 측정되었습니다.

종합 결론

전체 실험 결과, ① **결측치 처리 및 범주형 인코딩의 존재 자체** → ② **파생변수(특히 FarePerPerson, Title)** → ③ **Feature Selection** → ④ **스케일링** 순으로 생존 예측 성능에 미치는 영향이 컸습니다. 최종적으로 holdout 기준 최고 성능 조합은 **Mean Imputation + One-Hot Encoding + StandardScaler + 파생변수 + Logistic Regression (Accuracy 83.2%, F1 0.776, ROC-AUC 0.876)**이었으며, 이를 train.csv 전체로 재학습하여 Kaggle 제출용 submission.csv를 생성했습니다.

한계 및 향후 개선 방향

- 본 보고서의 수치는 단일 80/20 holdout split 기준으로, 표본 크기(179건)로 인한 변동성이 존재합니다. K-Fold 교차검증 평균으로 재검증 권장
- 실행 환경 제약으로 xgboost/lightgbm 대신 sklearn GradientBoosting을, SHAP 대신 Random Forest Feature Importance를 사용했습니다
- Cabin 원본 텍스트에서 deck(층) 정보를 추가로 추출하면 추가적인 성능 개선 여지가 있습니다

제출 자료 안내

소스코드: 전체 파이프라인 코드는 Titanic_ML_Pipeline.ipynb (Google Colab 실행 가능)에 정리되어 있으며, GitHub 저장소에 업로드 후 해당 링크로 제출합니다.

Kaggle 제출 파일: submission.csv (최종 모델로 test.csv 418건에 대한 생존 예측 포함)